
Fechete Serban Tudor
30431

**Motorcycle Service Web App
Supplementary Specification**

Version 1.0

Motorcycle Service Web App	Version: 1.0
Supplementary Specification	Date: 17/05/2026

Revision History

Date	Version	Description	Author
17.05.2026	1.0	Initial draft for Motorcycle Service Web App	Fechete Serban Tudor

Motorcycle Service Web App	Version: 1.0
Supplementary Specification	Date: 17/05/2026

Table of Contents

1.	Introduction	4
2.	Non-functional Requirements	4
2.1	Availability	4
2.2	Performance	4
2.3	Security	4
2.4	Testability	4
2.5	Usability	4
3.	Design Constraints	4

Motorcycle Service Web App	Version: 1.0
Supplementary Specification	Date: 17/05/2026

Supplementary Specification

1. Introduction

This Supplementary Specification captures the system requirements for the Motorcycle Service Web App that are not readily captured in the standard use cases. This includes critical quality attributes (such as strict JWT security, performance, and system availability), technological environments (Docker containerization), and architectural constraints (Angular and Spring Boot separation).

2. Non-functional Requirements

2.1 Availability

Source of stimulus: System Admin or hosting environment

Stimulus: Application container crash or server restart

Environment: Normal operation

Artifact: Docker containers and PostgreSQL Database

Response: The system restarts the containerized services quickly; database data is preserved externally.

Response measure: Minimal downtime; 0% data loss of customer profiles and appointments.

Tactics: Docker Containerization, Docker Volume Mapping to the host machine.

2.2 Performance

Source of stimulus: Customer or Admin

Stimulus: Requesting a large list of appointments or generating a PDF invoice

Environment: High system load

Artifact: Spring Boot Backend / Database Layer

Response: The system fetches data without loading unnecessary database relations into memory.

Response measure: API response time remains under 2 seconds.

Tactics: Hibernate ORM Lazy Loading strict transactional boundaries.

2.3 Security

Source of stimulus: Malicious actor or Unauthorized user

Stimulus: Attempting to access admin-only endpoints, guess passwords, or modify data

Environment: Public-facing internet

Artifact: Spring Security Configuration and jwt auth filter

Response: The system blocks the request entirely and logs the security event.

Response measure: 100% of unauthorized requests are rejected with a 401 or 403 status code.

Tactics: Stateless JWT Authentication, BCrypt Password Hashing, strict RBAC, End-to-End HTTPS/SSL.

2.4 Testability

Source of stimulus: Developer or QA

Stimulus: Need to verify complex business logic

Environment: Development

Artifact: Application Service Layer

Response: Tests can be run in isolation without needing to spin up the actual PostgreSQL database.

Response measure: High unit test coverage can be achieved quickly and reliably.

Tactics: Strict Layered Architecture allowing for easy Dependency Injection and mocking.

2.5 Usability

Source of stimulus: End User (Customer)

Stimulus: Navigating the dashboard to find and download a PDF invoice

Environment: Desktop or Mobile web browser

Artifact: Angular SPA (Frontend)

Motorcycle Service Web App	Version: 1.0
Supplementary Specification	Date: 17/05/2026

Response: The UI updates dynamically without full page reloads, providing a seamless experience.

Response measure: Complex tasks are completed in under 3 clicks without interrupting the user workflow.

Tactics: Single Page Application (SPA) architecture, asynchronous RxJS observables.

3. Design Constraints

The system must adhere to the following mandated design decisions and constraints:

- **Frontend Framework:** The client interface must be built using Angular (TypeScript).
- **Backend Framework:** The REST API must be built using Java 21 and the Spring Boot 3 framework.
- **Database:** Must utilize PostgreSQL as the underlying relational database, managed entirely via Hibernate ORM (no manual SQL scripts for schema creation).
- **Architecture:** Must adhere to a strict N-Tier/Layered architecture separating Presentation (Angular), Application Logic (Spring Boot Controllers/Services), and Data (Repositories).
- **Deployment:** The entire application stack (Frontend, Backend, DB) must be fully containerized and orchestratable via Docker Compose.
- **Third-Party Libraries:** The backend must utilize the iText7 library for server-side PDF generation.